

HOMEWORK 3

Kavya Saranya K V
2025115152

Q1. Write a C program to store n elements in an array using dynamic memory allocation and find the largest number.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i;
    printf("Enter n: ");
    scanf("%d", &n);

    int *arr = (int*)malloc(n * sizeof(int));

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int largest = arr[0];
    for (i = 1; i < n; i++)
        if (arr[i] > largest)
            largest = arr[i];

    printf("Largest = %d\n", largest);
    free(arr);
    return 0;
}
```

Sample Output:

```
Enter n: 5
Enter 5 elements: 12 45 7 89 23
Largest = 89
```

Q2. Print an array in reverse order using pointers.

Code:

```
#include <stdio.h>

int main() {
    int n, i;
    printf("Enter n: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int *ptr = arr + n - 1;
    printf("Reversed array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", *ptr);
        ptr--;
    }
    printf("\n");
    return 0;
}
```

Sample Output:

```
Enter n: 5
Enter 5 elements: 1 2 3 4 5
Reversed array: 5 4 3 2 1
```

Q3. Write a C program to store information of students using structure. Sort them by name.**Code:**

```
#include <stdio.h>
#include <string.h>

struct Student {
    char name[30];
    int roll;
    float marks;
};

int main() {
    int n, i, j;
    printf("Enter number of students: ");
    scanf("%d", &n);

    struct Student s[n];

    for (i = 0; i < n; i++) {
        printf("Enter name, roll, marks for student %d: ", i + 1);
        scanf("%s %d %f", s[i].name, &s[i].roll, &s[i].marks);
    }

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (strcmp(s[j].name, s[j + 1].name) > 0) {
                struct Student temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }

    printf("\nStudents sorted by name:\n");
    for (i = 0; i < n; i++)
        printf("%s %d %.2f\n", s[i].name, s[i].roll, s[i].marks);

    return 0;
}
```

Sample Output:

```
Enter number of students: 3
Enter name, roll, marks for student 1: Charlie 102 78.5
Enter name, roll, marks for student 2: Alice 101 92.0
Enter name, roll, marks for student 3: Bob 103 85.0

Students sorted by name:
Alice 101 92.00
Bob 103 85.00
Charlie 102 78.50
```

Q4. Write a C program to add n distances given in feet-inches system using structures.**Code:**

```
#include <stdio.h>

struct Distance {
```

```

        int feet;
        float inches;
    };

    int main() {
        int n, i;
        printf("Enter n: ");
        scanf("%d", &n);

        struct Distance d[n], total;
        total.feet = 0;
        total.inches = 0;

        for (i = 0; i < n; i++) {
            printf("Enter feet and inches for distance %d: ", i + 1);
            scanf("%d %f", &d[i].feet, &d[i].inches);
            total.feet += d[i].feet;
            total.inches += d[i].inches;
        }

        total.feet += (int)(total.inches / 12);
        total.inches = total.inches - (int)(total.inches / 12) * 12;

        printf("Total distance = %d feet %.2f inches\n", total.feet, total.inches);
        return 0;
    }

```

Sample Output:

```

Enter n: 2
Enter feet and inches for distance 1: 5 8
Enter feet and inches for distance 2: 3 7
Total distance = 9 feet 3.00 inches

```

Q5. Write a C program to add two complex numbers by passing structure to a function.

Code:

```

#include <stdio.h>

struct Complex {
    float real;
    float imag;
};

struct Complex add(struct Complex a, struct Complex b) {
    struct Complex sum;
    sum.real = a.real + b.real;
    sum.imag = a.imag + b.imag;
    return sum;
}

int main() {
    struct Complex c1, c2, result;

    printf("Enter real and imaginary parts of first complex number: ");
    scanf("%f %f", &c1.real, &c1.imag);

    printf("Enter real and imaginary parts of second complex number: ");
    scanf("%f %f", &c2.real, &c2.imag);

    result = add(c1, c2);

    printf("Sum = %.2f + %.2fi\n", result.real, result.imag);
    return 0;
}

```

Sample Output:

```
Enter real and imaginary parts of first complex number: 3 4
Enter real and imaginary parts of second complex number: 1 2
Sum = 4.00 + 6.00i
```

Q6. Write a C program to maintain an inventory management system using structures.**Code:**

```
#include <stdio.h>

struct Item {
    int id;
    char name[30];
    int quantity;
    float price;
};

int main() {
    int n, i;
    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item inv[n];

    for (i = 0; i < n; i++) {
        printf("Enter id, name, quantity, price for item %d: ", i + 1);
        scanf("%d %s %d %f", &inv[i].id, inv[i].name, &inv[i].quantity, &inv[i].price);
    }

    printf("\nInventory:\n");
    printf("ID\tName\tQty\tPrice\tTotal\n");
    for (i = 0; i < n; i++)
        printf("%d\t%s\t%d\t%.2f\t%.2f\n", inv[i].id, inv[i].name, inv[i].quantity,
            inv[i].price, inv[i].quantity * inv[i].price);

    return 0;
}
```

Sample Output:

```
Enter number of items: 2
Enter id, name, quantity, price for item 1: 101 Pen 50 10.00
Enter id, name, quantity, price for item 2: 102 Notebook 30 25.50

Inventory:
ID Name      Qty Price Total
101 Pen      50  10.00 500.00
102 Notebook 30  25.50 765.00
```

Q7. Write a C program for decimal to binary, octal and hexadecimal conversion.**Code:**

```
#include <stdio.h>

int main() {
    int n, temp, i;
    printf("Enter a decimal number: ");
    scanf("%d", &n);

    temp = n;
    int bin[32], count = 0;
    if (temp == 0) bin[count++] = 0;
    while (temp > 0) {
        bin[count++] = temp % 2;
        temp = temp / 2;
    }
```

```

        temp /= 2;
    }
    printf("Binary: ");
    for (i = count - 1; i >= 0; i--)
        printf("%d", bin[i]);
    printf("\n");

    printf("Octal: %o\n", n);
    printf("Hexadecimal: %x\n", n);

    return 0;
}

```

Sample Output:

```

Enter a decimal number: 25
Binary: 11001
Octal: 31
Hexadecimal: 19

```

Q8. Write a C program to calculate the difference between two time periods.

Code:

```

#include <stdio.h>

struct Time {
    int hours;
    int minutes;
    int seconds;
};

int main() {
    struct Time t1, t2, diff;

    printf("Enter first time (hh mm ss): ");
    scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);

    printf("Enter second time (hh mm ss): ");
    scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);

    int total1 = t1.hours * 3600 + t1.minutes * 60 + t1.seconds;
    int total2 = t2.hours * 3600 + t2.minutes * 60 + t2.seconds;

    int diffSec = total1 > total2 ? total1 - total2 : total2 - total1;

    diff.hours = diffSec / 3600;
    diff.minutes = (diffSec % 3600) / 60;
    diff.seconds = diffSec % 60;

    printf("Time difference = %d:%d:%d\n", diff.hours, diff.minutes, diff.seconds);
    return 0;
}

```

Sample Output:

```

Enter first time (hh mm ss): 10 30 45
Enter second time (hh mm ss): 8 15 20
Time difference = 2:15:25

```

Q9. Write a C program to implement an employee management system using structures.

Code:

```

#include <stdio.h>

struct Employee {
    int id;

```

```

    char name[30];
    char designation[20];
    float salary;
};

int main() {
    int n, i;
    printf("Enter number of employees: ");
    scanf("%d", &n);

    struct Employee emp[n];

    for (i = 0; i < n; i++) {
        printf("Enter id, name, designation, salary for employee %d: ", i + 1);
        scanf("%d %s %s %f", &emp[i].id, emp[i].name, emp[i].designation, &emp[i].salary);
    }

    printf("\nEmployee Records:\n");
    printf("ID\tName\tDesignation\tSalary\n");
    for (i = 0; i < n; i++)
        printf("%d\t%s\t%s\t%.2f\n", emp[i].id, emp[i].name, emp[i].designation, emp[i].salary);

    return 0;
}

```

Sample Output:

```

Enter number of employees: 2
Enter id, name, designation, salary for employee 1: 201 John Manager 55000
Enter id, name, designation, salary for employee 2: 202 Sara Developer 45000

Employee Records:
ID Name Designation Salary
201 John Manager 55000.00
202 Sara Developer 45000.00

```

Q10. Write a C program to calculate area and perimeter of a rectangle and triangle using function pointers.

Code:

```

#include <stdio.h>
#include <math.h>

float rectArea(float l, float b) {
    return l * b;
}

float rectPerimeter(float l, float b) {
    return 2 * (l + b);
}

float triArea(float a, float b, float c) {
    float s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}

float triPerimeter(float a, float b, float c) {
    return a + b + c;
}

int main() {
    float l, b, a1, b1, c1;

    float (*rectFunc[2])(float, float) = {rectArea, rectPerimeter};
    printf("Enter length and breadth of rectangle: ");
}

```

```
scanf("%f %f", &l, &b);
printf("Rectangle Area = %.2f\n", rectFunc[0](l, b));
printf("Rectangle Perimeter = %.2f\n", rectFunc[1](l, b));

float (*triFunc[2])(float, float, float) = {triArea, triPerimeter};

printf("Enter three sides of triangle: ");
scanf("%f %f %f", &a1, &b1, &c1);
printf("Triangle Area = %.2f\n", triFunc[0](a1, b1, c1));
printf("Triangle Perimeter = %.2f\n", triFunc[1](a1, b1, c1));

return 0;
}
```

Sample Output:

```
Enter length and breadth of rectangle: 5 3
Rectangle Area = 15.00
Rectangle Perimeter = 16.00
Enter three sides of triangle: 3 4 5
Triangle Area = 6.00
Triangle Perimeter = 12.00
```